

gas_db

Data Warehouse — Knowledge Transfer Documentation

Azure Cosmos DB for PostgreSQL (Citrus) · ADF Medallion Pipeline · Grubrrr Analytics Platform

Prepared by: Nodir
Data Warehouse Engineer, Grubrrr

July 2026

Prepared as part of departure knowledge-transfer to the engineers assuming gas_db ownership

Table of Contents

Table of Contents	2
1. Purpose & Scope	4
2. System Overview	4
2.1 Database Inventory Summary	5
3. Schema: dim — Reference & Dimension Data	6
3.1 Table Inventory	6
3.2 Standard dim Refresh Pattern	7
3.3 Stored Procedures (20)	8
3.4 Notable Special Cases	9
4. Schema: fact — Transactional & Event Data	10
4.1 Table Inventory	10
4.2 Stored Procedures & Functions (36)	11
4.3 Walkthrough: fact.usp_silver_transaction_header_to_fact	13
4.4 fact.deviceevent — Deduplication & Performance Project	14
5. Schema: ml — Machine Learning Feature Tables	15
5.1 Table Inventory	15
5.2 Stored Procedures (8)	15
5.3 Fuzzy Item–Modifier Matching Subsystem	16
6. Schema: stg — Silver Staging Layer	17
Dimension staging (loaded by ADF Copy/Data Flow from Cosmos DB, consumed by dim.usp_refresh_*)	17
Device/infrastructure staging (full-truncate each run — see 6.2)	17
Orders/transaction pipeline staging (10-minute sliding-window truncate — see 6.2)	18
Legacy / working / lookup tables	18
6.1 The camelCase Convention	19
7. Schema: etl — Pipeline Orchestration Metadata	19
7.1 etl.bronze_partition_registry	19
7.2 etl.truncate_silver_staging()	20
8. Sequences — Manual Surrogate Key Standard	21
9. Indexes	21
9.1 BRIN Coverage — Known Gap	22
9.2 Partial Indexes	22
10. Cross-Cutting Stored-Procedure Conventions	22
10.1 Definition & ownership boilerplate	22
10.2 Watermark pattern	23
10.3 Dedup: DISTINCT ON over ROW_NUMBER()	23
10.4 Temp table + ANALYZE before merge	23

10.5 Non-destructive upsert: ON CONFLICT DO UPDATE + COALESCE + guard	23
10.6 Manual sequence-based surrogate keys.....	23
10.7 Session-parameter tuning for large-batch loops	23
11. ADF Data Factory — Orchestration Layer Overview.....	25
11.1 Linked Services (15).....	25
11.2 Triggers (4).....	26
12. Master Orchestration: DimensionsFactsMaster	28
12.1 Success Path	28
12.2 Failure Branches.....	28
12.3 Sub-Pipeline Call Tree	28
13. Parallel Hour-Partition Read Pattern	29
13.1 Phase 1 — Parallel Existence Check	29
13.2 Phase 2 — Sequential Load.....	29
14. Known Issues, Landmines & Recommended Follow-ups	31
14.1 High Blast Radius.....	31
14.2 Verify Against Production (DDL Export May Be Stale).....	31
14.3 Known Data-Quality / Design Gaps.....	31
14.4 Monitoring Gap	32
15. Recommended KT Session Plan	32
16. Appendices.....	33
Appendix A — Full ADF Pipeline Inventory (67, alphabetical)	33
Appendix B — Source File Cross-Reference	34
Appendix C — Document Change Log	34

1. Purpose & Scope

This document is the primary knowledge-transfer (KT) artifact for gas_db, the PostgreSQL analytics data warehouse built to serve Grubrr's kiosk, ordering, device-telemetry, and ML/DS reporting needs. It is written for the engineer(s) assuming ownership of gas_db and its Azure Data Factory (ADF) orchestration layer, and is meant to be read alongside the nine source artifacts it was generated from:

- gas_db_ddls_merged_20260530.sql — full merged DDL export (128 tables across 5 schemas)
- gas_indexes.sql — all index and constraint definitions (105 index statements)
- gas_db_sequence_synchronization.sql — the 12 manually-managed surrogate-key sequences
- dim_stored_procs.sql, fact_stored_procs.sql, ml_stored_procs.sql — 64 stored procedures/functions
- etl_bronze_partition_registry.sql — the Bronze partition-watermark table + working queries
- silver_usp_etl_truncate_silver_staging.sql — the Silver staging truncate/window procedure
- ARMTemplateForFactory.json — the full ADF ARM export (67 pipelines, 80 data flows, 198 datasets, 15 linked services, 4 triggers)

HOW THIS DOCUMENT WAS BUILT: This document was generated by analyzing the nine uploaded source files directly (schema parsing, stored-procedure inspection, and ADF pipeline-dependency traversal), cross-checked against prior working context. Section 15 flags a handful of places where the uploaded DDL snapshot appears to be behind what is understood to be true of the live database — these are called out explicitly so they get verified against production rather than assumed.

2. System Overview

gas_db is the Gold-layer analytics warehouse at the end of a medallion pipeline that starts in operational Cosmos DB instances and ends in reporting-ready PostgreSQL fact/dim tables. The high-level flow:

```

Cosmos DB (NGE orders, GEM events, GXS/GOU, GSH device telemetry)
|
▼ ADF Copy Activities (per-hour partitions)
ADLS Gen2 – Bronze layer
(Hive-style partitioning: {entity}/raw/YYYY/MM/DD/HH, NDJSON, hourly folders)
|
▼ ADF Copy Activities / Data Flows
stg schema (PostgreSQL) – Silver staging
(silver_* tables – landing zone, camelCase-quoted JSON-mirroring columns,
10-minute sliding window for event-joined tables, full truncate for others)
|
▼ PL/pgSQL stored procedures (usp_silver_*_to_fact, usp_refresh_*)
dim / fact / ml schemas (PostgreSQL) – Gold
(surrogate-keyed dimensions, watermarked/deduped facts, ML feature tables)

```

The platform is Azure Cosmos DB for PostgreSQL (Citrus) — a distributed PostgreSQL service. gas_db is not a data-lakehouse-only design: it deliberately keeps a second, fully-relational copy of Gold data in PostgreSQL (rather than

querying Parquet directly) specifically to retain PL/pgSQL stored procedures, ACID multi-statement transactions, and custom functions such as `dim.token_sort_ratio`, which a pure lakehouse/Spark-SQL engine would not support as cleanly. This hybrid tradeoff — paying for two copies of Gold-grain data in exchange for stored-procedure-based ETL and transactional upserts — is a deliberate architecture decision, not an oversight, and should be preserved by whoever inherits the system unless the tradeoff is revisited.

2.1 Database Inventory Summary

Schema	# Tables	# Stored Procs/Fns	Purpose
dim	41	20	Slowly-changing reference/dimension data: menu, catalog, kiosk, location, organization, customer.
fact	33	36 (34 proc + 2 fn)	Transactional and event-grain measures: orders, device events, telemetry, surveys, recommendations.
ml	8	8	Denormalized feature tables consumed by the ML/Data Science team.
stg	45	0 (loaded by ADF Copy/Data Flow, truncated by <code>etl.truncate_silver_staging</code>)	Silver staging / landing zone for Bronze data before Gold load.
etl	1	1 (<code>truncate_silver_staging</code>)	Pipeline orchestration metadata: Bronze partition watermark registry.

128 tables and 64 stored routines total, orchestrated by 67 ADF pipelines. The sections below cover each schema in depth, then the ADF orchestration layer, then a consolidated list of known issues and a recommended KT session plan.

3. Schema: dim — Reference & Dimension Data

41 tables. Slowly-changing reference data refreshed from stg.dim_* staging tables. Every dim refresh procedure follows the same rebuild pattern (Section 3.2) and every surrogate-keyed dim table uses one of the 12 manually-managed sequences described in Section 8, not IDENTITY columns.

3.1 Table Inventory

Table	#Cols	Purpose
abtests	12	A/B test assignment per order session — links experimentid/variantid to a device and session.
catalog	14	One row per menu catalog (a catalog is the top-level menu container tied to an organization).
category_hierarchy	29	Widest dim table (29 cols) — full catalog→category→subcategory mapping hierarchy incl. audit timestamps.
company	10	Company/brand-level reference data (business contact info, address).
datedim	13	Standard calendar date dimension (day/week/month/quarter/year attributes) — joined by nearly every fact table for dateid.
device	15	Physical device registry (kiosks, KDS, etc.) with current/previous operational state.
duplicate_items_master	12	Output of dim.usp_master_keys_for_duplicate_items — maps duplicate item instances to one master item key per category.
element	5	"GOU element" lookup; surrogate key via dim.element_elementid_seq.
experiment	2	Minimal experiment key/data pair supporting dim.abtests.
feedbackrating	2	Static lookup: survey rating code → description.
feedbackstatus	2	Static lookup: survey transaction status code → description.
frequentcustomer	15	Loyalty/repeat-customer dimension (name, contact, first/last order date).
frequentcustomer_bkp	15	Point-in-time backup of dim.frequentcustomer (pre-refresh safety copy).
grubrrr_source_lookup	3	Static lookup of upstream source systems (NGE/GEM/GSH/GXS/GOU) referenced by sourceid columns.
holidays	8	Holiday calendar (name, date, religion/type flags) used for demand-pattern and weather-adjacent analysis.
item_modifier_group_modifier_mapping	25	Item→modifier-group→modifier bridge with min/max/free-count selection rules (25 cols).
itemcategory	17	Location-scoped category dimension.
itemcategory_bkp	5	Backup copy of dim.itemcategory.
itemcategorymapping	9	Category-to-menu-item mapping bridge table.
kiosk	12	Kiosk device dimension (serial number, app version, test-kiosk flag); surrogate key via dim.kiosk_id_seq.

Table	#Cols	Purpose
kioskdetails	42	Widest kiosk-related dim (42 cols) — sync version, POS/loyalty/payment provider linkage per kiosk.
location	14	Store/location master (address, geo-coordinates, grouping).
locationcatalog	7	Location-to-catalog assignment, including timezone — the timezone column is joined constantly for UTC→local conversions.
menuentities	26	Generic catalog entity dimension (items, combos, categories) with nutrition attributes (26 cols).
menuitem	26	Core menu item dimension (26 cols) — item class, nutrition facts; surrogate key via dim.menuitem_id_seq. Joined by nearly every fact-load procedure.
modifier	21	Modifier dimension (quantity rules, calories, active/deleted flags).
modifier_group	21	Modifier group dimension (min/max/free selection counts).
modifier_group_mapping	18	Modifier-to-modifier-group bridge with default-selection flags.
occasionsurvey	13	Survey definition dimension (type, question/selection type, status); surrogate key via dim.occasionsurvey_surveykey_seq.
ordertype	7	Order-type lookup scoped by location + kiosk; surrogate key via dim.ordertype_id_seq. Joined during every transactionheader load.
ordertype_bkp	5	Backup copy of dim.ordertype.
organization	34	Widest core dim (34 cols) — the company/org/location hierarchy root; almost every fact/dim procedure joins this for timezone and org attributes.
organizationlocation	8	Organization-to-location bridge, incl. round-up-for-charity flag.
peripheral	11	Peripheral device dimension (printers, scanners, payment terminals) with state history.
upsellgrouplookup	7	Upsell group lookup used by the recommendation/upsell fact pipeline.
userlocation	3	User-to-location access mapping.
view	4	"GOU view" lookup; surrogate key via dim.view_id_seq.
vw_grubrrinstallbase	142	Materialized install-base reporting table (142 cols) — rebuilt by dim.usp_grubrr_install_base; one row per active kiosk/device with full org/location/hardware/software context.
vw_grubrrinstallbase_all_devices	142	Same shape as vw_grubrrinstallbase but includes inactive/retired devices; rebuilt by dim.usp_grubrr_install_base_all_devices.
weather	9	Current weather-info dimension per organization/location (raw API payload + parsed fields).
weather_bkp	8	Backup copy of dim.weather, keyed by businessdate.

3.2 Standard dim Refresh Pattern

All 15 dim.usp_refresh_* procedures (menuitem, kiosk, organization, modifier, modifiergroup, catalog, category_hierarchy, frequentcustomer, itemcategory, occasionsurvey, ordertype, organizationlocation, element, view, dim_location_kiosk_details) share one pattern:

```
CREATE TEMP TABLE tmp_<entity> ON COMMIT DROP AS — pull + shape the delta from stg.dim_<entity>
→ ANALYZE tmp_<entity> — refresh planner statistics on the temp table before it drives a join/merge
→ INSERT INTO dim.<entity> ... SELECT ... FROM tmp_<entity> ON CONFLICT (...) DO UPDATE SET ... — upsert
against the natural key
→ nextval('dim.<entity>_..._seq') supplies the surrogate id column on INSERT — never an IDENTITY default
```

Because ON COMMIT DROP temp tables are transaction-scoped, and because Citus distributes tables across worker shards, this pattern keeps each refresh procedure self-contained and idempotent — safe to re-run without manual cleanup if a run fails partway.

3.3 Stored Procedures (20)

Procedure	Purpose
usp_grubrr_install_base	Rebuilds dim.vw_grubrrinstallbase — the full device/install-base reporting view (142 cols) joining device, location, org, kiosk, and peripheral dimensions.
usp_grubrr_install_base_all_devices	Variant of the install-base rebuild that includes all devices (not just active/in-service units).
usp_master_keys_for_duplicate_items	Resolves dim.duplicate_items_master — maps duplicate menu-item records (across catalogs/locations) to a single master item key.
usp_refresh_catalog	Temp-table rebuild + upsert of dim.catalog from stg.dim_catalog.
usp_refresh_category_hierarchy	Temp-table rebuild + upsert of dim.category_hierarchy (29 cols) from stg.dim_category_hierarchy; the pipeline previously implicated in the ADF SSL/connection-exhaustion incident.
usp_refresh_dim_location_kiosk_details	Refreshes a combined location+kiosk detail dimension used for install-base and kiosk reporting.
usp_refresh_element	Refreshes dim.element (GOU "element" lookup) — uses dim.element_elementid_seq for surrogate keys.
usp_refresh_frequentcustomer	Temp-table rebuild + upsert of dim.frequentcustomer (loyalty/repeat-customer dimension) from stg.dim_frequentcustomer.
usp_refresh_itemcategory	Temp-table rebuild + upsert of dim.itemcategory from stg.dim_itemcategory.
usp_refresh_kiosk	Temp-table rebuild + upsert of dim.kiosk from stg.dim_kiosk.
usp_refresh_location_kiosk_details	Refreshes location-to-kiosk mapping details from stg.dim_location_kiosks.
usp_refresh_menuitem	Temp-table rebuild + upsert of dim.menuitem (26 cols) from stg.dim_menuitem — one of the highest-traffic dimension refreshes (joined by nearly every fact-load procedure).
usp_refresh_modifier	Temp-table rebuild + upsert of dim.modifier from stg.dim_modifier.
usp_refresh_modifiergroup	Temp-table rebuild + upsert of dim.modifier_group from stg.dim_modifiergroup.

Procedure	Purpose
usp_refresh_modifiergroup_modifier_mapping	Refreshes dim.modifier_group_mapping (the modifier-group-to-modifier bridge table) from stg.dim_modifiergroup_modifier_mapping.
usp_refresh_occasionsurvey	Temp-table rebuild + upsert of dim.occasionsurvey from stg.dim_occasionsurvey.
usp_refresh_ordertype	Temp-table rebuild + upsert of dim.ordertype from stg staging; feeds the ordertype lookup joined during transactionheader loads.
usp_refresh_organization	Temp-table rebuild + upsert of dim.organization (34 cols — the company/org/location hierarchy root) from stg.dim_organization.
usp_refresh_organizationlocation	Temp-table rebuild + upsert of dim.organizationlocation from stg.dim_organizationlocation.
usp_refresh_view	Refreshes dim.view (GOU "view" lookup) — uses dim.view_id_seq for surrogate keys.

3.4 Notable Special Cases

- dim.usp_master_keys_for_duplicate_items — not a simple refresh; it resolves duplicate menu-item instances across categories/locations to a single dim.duplicate_items_master.masteritemid. This feeds downstream dedup logic elsewhere in the warehouse — trace its consumers before changing its output shape.
- dim.usp_grubrr_install_base / _all_devices — these rebuild 142-column reporting views used directly by the install-base dashboard (see ADF trigger TwiceDaily in Section 12.4). They are wide joins across device/kiosk/location/org/peripheral — check execution time before scheduling them more frequently.
- dim.organization and dim.menuitem are the two most heavily-joined dim tables in the entire warehouse — any schema change to either has a wide blast radius across both dim and fact procedures.

4. Schema: fact — Transactional & Event Data

33 tables, 34 load/aggregation procedures, and 2 utility functions. This is where order, device-event, telemetry, survey, and recommendation grain data lands after Silver staging. Two utility tables underpin the whole schema: fact.watermarktable (read by nearly every silver-to-fact loader to bound its delta) and fact.pipelinerrunstatus (written by the ADF master pipelines — see Section 13).

4.1 Table Inventory

Table	#Cols	Purpose
cep_incidents	19	CEP (Complex Event Processing) incident/alert facts raised by the connector/device layer.
customer_menu_preferences	9	Per-customer daypart × item ordering-frequency aggregation (rebuilt by fact.usp_customer_menu_preferences).
deviceevent	20	Largest fact table in the warehouse (262M+ rows) — raw kiosk/device event stream (session, order, payment, insight events). Recently deduplicated and performance-tuned (Section 4.3).
devicehealth	13	Device health-check status events.
devicestate	15	Device state transitions with duration; source for fact.updatewatermark(); surrogate key via fact.devicestate_id_seq.
devicetelemetry	10	CPU/memory telemetry samples per device.
gem_failed_order_job_notifications	15	Operational-monitoring fact for failed order-processing jobs (incident count, first/last occurrence).
itemmodifier	14	Order-line-item-to-modifier fact (which modifiers were attached to which item on which order).
itemssurvey	22	Item-level post-order survey rating fact.
location_menu_preferences	8	Location-level daypart × item ordering-frequency aggregation.
location_statistics	22	Location-grain KPI rollup (22 cols) — org/location attributes joined with order-type performance labels.
modifier_impressions	23	Every modifier shown to a customer during ordering (impression-level, pre-selection) — 23 cols incl. nesting depth/position for menu-UX analysis.
modifier_interactions	24	Every modifier the customer actually interacted with/selected (24 cols) — pairs with modifier_impressions for funnel/conversion analysis.
modifier_recommendations	13	Recommendation-engine modifier suggestions and outcome tracking, tied back to impressions/interactions.
occasionsurveydetail	16	Order-level (not item-level) post-order survey response fact.
ordertiming	27	Millisecond-level kiosk-session timing checkpoints (session/menu/item/checkout starts) — 27 cols; surrogate key via fact.ordertiming_id_seq.
peripheralhealth	5	Peripheral (printer/scanner/terminal) health-check status.
peripheralstate	7	Peripheral state transitions with duration.

Table	#Cols	Purpose
pipelinerunstatus	9	ADF master-pipeline run log — written by DimensionsFactsMaster (and child masters) on both success and failure. See Section 13.
pos_sales_details	11	POS-reported daily sales/tips reconciliation fact.
recommendations	9	Item-recommendation offer/acceptance fact (offered vs. selected items, conversion flag).
recommendations_bkp	7	Backup copy of fact.recommendations.
sent_surveys	12	Survey-send event log (which survey was sent for which order, GEM event metadata).
timingsdatalake	2	Minimal 2-column timing metric sink (container name + timing value) — likely an ADF/pipeline-duration probe table.
transactionheader	45	Core order-header fact (45 cols) — one row per completed order; the most heavily-modeled fact table in the warehouse. Surrogate key via fact.transactionheader_id_seq.
transactionitem	29	Order line-item fact (29 cols) — one row per item/combo on an order.
transactionpayment	14	Order payment fact (tender type, amount, integration label).
transactionrefunds	10	Order refund fact, linked back to the original transaction and payment.
userbehaviour	20	Kiosk session/clickstream behavior fact (20 cols) — recently rebuilt via INSERT...SELECT with enrichment joins during the dedup/performance cycle. Surrogate key via fact.userbehaviour_id_seq.
userbehaviour_exceptions	13	Rows that failed userbehaviour enrichment/validation, quarantined for investigation rather than silently dropped.
usercheckedin	18	Customer check-in event fact (dine-in / call-ahead style flows).
vw_offer_analysis	19	Upsell-offer performance fact (offered vs. selected item, upsell group, quantity).
watermarktable	8	The central ETL watermark table — one row per (watermarktablename, source) pair holding ticks/ts high-water marks read by nearly every silver-to-fact loader.

4.2 Stored Procedures & Functions (36)

Two utility functions, plus 34 procedures that fall into two groups: (a) silver-to-fact loaders that move a Silver staging table into its Gold fact table, and (b) analytics/aggregation procedures that recompute a derived fact from data already in Gold.

Procedure / Function	Purpose
fn_getdata	Utility TABLE function — parameterized lookup (application, data category, module, action type) over device-event-shaped data.
updatewatermark	Utility function that stamps fact.watermarktable.watermarkvalue with MAX(fact.devicestate.healthdatatime) for the given watermark key.
usp_customer_menu_preferences	Rebuilds fact.customer_menu_preferences — per-customer daypart/item ordering-frequency aggregation from fact.transactionheader/transactionitem.

Procedure / Function	Purpose
usp_distribute_silver_gem_events	Fan-out procedure that routes stg.silver_kiosk_events / stg.silver_all_gem_events rows to the correct fact loaders by event category.
usp_distribute_silver_transaction_entities	Fan-out procedure that routes staged transaction-entity rows (header/item/payment/combo/etc.) to their respective fact loaders.
usp_gem_ordertiming_to_fact_ordertiming	Loads GEM order-timing telemetry into fact.ordertiming (27 cols).
usp_gem_sent_surveys_to_fact	Loads staged sent-survey events into fact.sent_surveys.
usp_gem_usercheckedin_to_fact_usercheckedin	Loads GEM user-check-in events into fact.usercheckedin.
usp_gsh_devicehealth_to_fact_devicestate	Loads GSH device-health telemetry into fact.devicestate; source for fact.updatewatermark().
usp_gsh_devicetelemetry_to_fact_devicetelemetry	Loads GSH device telemetry into fact.devicetelemetry.
usp_item_recommendations_stage_to_fact	Loads staged item-recommendation rows into fact.recommendations.
usp_location_menu_preferences	Rebuilds fact.location_menu_preferences — location-level daypart/item ordering aggregation.
usp_location_statistics	Rebuilds fact.location_statistics (22 cols) — location-grain KPI rollups.
usp_modifier_impression_analysis	Aggregates stg.silver_modifier_impressions into fact.modifier_impressions (23 cols).
usp_modifier_interaction_analysis	Aggregates stg.silver_modifier_interactions into fact.modifier_interactions (24 cols).
usp_modifier_recommendation_analysis	Aggregates stg.silver_modifier_recommendations into fact.modifier_recommendations.
usp_nge_update_itemssurvey	Applies NGE-sourced updates to fact.itemssurvey rows already loaded.
usp_offer_analysis	Rebuilds fact.vw_offer_analysis (19 cols) — offer/promotion performance aggregation.
usp_sent_surveys_to_fact_itemssurvey	Loads staged sent-survey data into fact.itemssurvey.
usp_silver_aborted_orders_and_items_to_fact	Loads abandoned/aborted kiosk sessions (never completed to a transaction) into their fact destination for funnel analysis.
usp_silver_item_modifiers_to_fact	Loads stg.silver_item_modifiers (52 cols, the widest staging table) into fact.itemmodifier.
usp_silver_kiosk_events_to_fact_deviceevent	Loads stg.silver_kiosk_events into fact.deviceevent — the largest fact table (262M+ rows); dedup/performance-tuned via ROW_NUMBER loop with mid-loop commits (see 12.3).
usp_silver_kiosk_events_to_fact_userbehaviour	Loads stg.silver_kiosk_events into fact.userbehaviour (20 cols) — the session/clickstream behavior fact, rebuilt via INSERT...SELECT with enrichment joins during the recent performance cycle.
usp_stg_gem_failed_order_job_notifications_to_fact	Loads failed-order-job notification events into their fact table for operational monitoring.
usp_silver_cep_incidents_to_fact_cep_incidents	Loads stg.silver_cep_incidents into fact.cep_incidents (19 cols) — CEP (Complex Event Processing) incident/alert facts.
usp_silver_modifier_impressions_to_fact	Loads stg.silver_modifier_impressions into fact.modifier_impressions.

Procedure / Function	Purpose
usp_silver_modifier_interactions_to_fact	Loads stg.silver_modifier_interactions into fact.modifier_interactions.
usp_silver_modifier_recommendations_to_fact	Loads stg.silver_modifier_recommendations into fact.modifier_recommendations.
usp_silver_transaction_header_to_fact	Loads stg.silver_transaction_header into fact.transactionheader (45 cols) — watermark filter, DISTINCT ON dedup, kiosk-event journey-timing enrichment join, INSERT...ON CONFLICT DO UPDATE with COALESCE guards. See walkthrough in 12.2.
usp_silver_transaction_item_to_fact	Loads stg.silver_transaction_item into fact.transactionitem (29 cols), the same pattern as the header loader.
usp_silver_transaction_payment_to_fact	Loads stg.silver_transaction_payment into fact.transactionpayment.
usp_silver_transaction_refunds_to_fact	Loads stg.silver_transaction_refunds into fact.transactionrefunds.
usp_silver_upsell_recommendations_to_fact	Loads stg.silver_upsell_recommendations into fact.recommendations.
usp_stg_occasionsurveydetail_to_fact	Loads stg.fact_occasionsurveydetail into fact.occasionsurveydetail (16 cols).
usp_update_datetime_fields	Backfill/repair procedure that re-parses and corrects datetime columns across fact tables (ties to the to_timestamp/HH12-AM-PM parsing work).
usp_update_occasion_survey_datetime_fields	Same datetime backfill/repair pattern, scoped to fact.occasionsurveydetail.

4.3 Walkthrough: fact.usp_silver_transaction_header_to_fact

This procedure is the clearest example of the full silver-to-fact loading pattern used across the warehouse (transactionitem, transactionpayment, transactionrefunds, deviceevent, and userbehaviour loaders all follow the same shape). Reading it top-to-bottom:

Capture the watermark once, upfront: `SELECT COALESCE(ts, <seed>) - 600 FROM fact.watermarktable WHERE watermarktablename = 'fact.transactionheader' AND source = 'nge'` — the 600-second (10-minute) buffer exists for the same reason `etl.truncate_silver_staging` keeps a 10-minute sliding window (Section 6.2): late-arriving events near an hour boundary.

`delta_transactions` CTE — `SELECT DISTINCT ON (locationid, transactionheaderid) ... ORDER BY locationid, transactionheaderid, orderdateutc DESC` pulls the latest version of each transaction. Note the commented-out line `AND syscosmosts > v_max_syscosmosts`, with an inline comment recording why it was removed: "cross-hour syscosmosts values can't always be reliable" — this is the watermark-gate bug fix.

`qualified_trxns` CTE — LEFT JOINs `dim.ordertype` and `dim.organization` to resolve `ordertype` and convert `orderdateutc` to `orderdatelocal` using the location's timezone.

`aggregated_kiosk_events` CTE — pulls only `stg.silver_kiosk_events` rows matching sessions already present in this batch (inner-joined to `qualified_trxns` first) specifically to avoid a full scan of `silver_kiosk_events` on every run, then pivots `session/menu/checkout/payment` timestamps out with `MIN()/MAX()` CASE expressions.

`orders_enriched` CTE — second DISTINCT ON pass joining the pivoted kiosk-event timings onto the transaction.

`INSERT INTO fact.transactionheader (...)` `SELECT nextval('fact.transactionheader_id_seq') AS id, ... FROM orders_enriched` — manual sequence-generated surrogate key, never IDENTITY.

`ON CONFLICT (locationid, transactionheaderid) DO UPDATE SET ...` using `COALESCE(fact.transactionheader.col, EXCLUDED.col)` for every timing column, so a later run only fills in columns that are still NULL — it never clobbers a timestamp that was already captured.

WHERE (...) guard on the DO UPDATE — an explicit IS NULL / IS NOT NULL check per column so the UPDATE is skipped entirely (no-op, no updateddate bump) when nothing would actually change.

PATTERN TO PRESERVE: If you are asked to change transaction-timing logic, change it in the CTEs (delta_transactions / qualified_trxns / aggregated_kiosk_events / orders_enriched) — not in the ON CONFLICT clause. The ON CONFLICT clause is deliberately additive/non-destructive; loosening its COALESCE guards to always overwrite would let a late-arriving partial re-run erase timing data that a previous run already captured correctly.

4.4 fact.deviceevent — Deduplication & Performance Project

fact.deviceevent (262M+ rows) and fact.userbehaviour were the subject of a dedicated dedup/performance engineering cycle: a per-location loop procedure using ROW_NUMBER() with mid-loop COMMITs (to avoid one giant long-running transaction against a 262M-row table), a new unique constraint added to fact.deviceevent to make future loads naturally idempotent, and a full rebuild of fact.userbehaviour via INSERT...SELECT with enrichment joins. Session-level tuning found to matter most:

- synchronous_commit = OFF — the single biggest win for a loop doing many small commits.
- jit = OFF — JIT compilation overhead was not worth it for many short, repeated executions.
- work_mem tuned per workload — 1GB for the dedup loop, 256MB for the fact.userbehaviour rebuild.

VERIFY AGAINST PRODUCTION: The uploaded DDL export (gas_db_ddls_merged_20260530.sql) does not show a unique constraint on fact.deviceevent, and gas_indexes.sql shows the deviceeventidx composite index as commented out (removed for zero usage) with the BRIN index on fact.deviceevent.syscosmots also commented out (not yet created). If the dedup project's unique constraint and any accompanying index were added after May 30, 2026, they will not be reflected in these files — the incoming engineer should confirm current state directly against production (\d fact.deviceevent) rather than trusting this DDL snapshot for this specific table.

The per-location loop script itself was run as an ad-hoc maintenance operation and is not part of the committed stored-procedure files reviewed for this document. If it still exists (as a saved .sql script, a notebook, or in shell history), it should be preserved and handed off explicitly — it is not discoverable from the DDL/stored-proc exports alone.

5. Schema: ml — Machine Learning Feature Tables

8 tables, 8 procedures. This schema exists specifically to hand the ML/Data Science team clean, business-aggregated, denormalized data without exposing them to raw fact/dim joins — every ml table spells out organization/location names rather than leaving them as IDs, and most add calendar-part columns (yyyy, mm, dd, ww, hh) for time-series feature engineering.

5.1 Table Inventory

Table	#Cols	Purpose
item_modifiergroup_modifier_mapping	28	Denormalized, org/location-named item→modifier-group→modifier mapping (28 cols) — the ML-facing counterpart of dim.item_modifier_group_modifier_mapping, with names spelled out rather than left as IDs for feature readability.
menu_entities	28	Flattened menu-item + category feature table (28 cols) with unit price and class-type attributes, handed to the ML/DS team for item-level modeling.
modifier_impressions	33	ML feature version of fact.modifier_impressions (33 cols) — adds calendar features (yyyy, ww) for time-series modeling.
modifier_interactions	38	ML feature version of fact.modifier_interactions (38 cols, the widest ml table) — same calendar-feature enrichment.
modifier_item_match	33	Output of the fuzzy item–modifier matching subsystem (Section 5.3) — matched_menuitemid, tsr_score, match_confidence_tier, match_direction per modifier.
transactions	37	Order-line-grain feature table (37 cols) built from fact.transactionheader/transactionitem, denormalized with organization/location names for direct ML consumption.
upsell_analysis	20	ML feature version of fact.vw_offer_analysis — upsell offer/acceptance features by customer and item class.
weather	59	Widest ml table (59 cols) — weather features broken out by yyyy/mm/dd/ww/hh granularity, joined to location/date for demand-modeling features.

5.2 Stored Procedures (8)

Procedure	Purpose
usp_refresh_item_modifiergroup_modifier_mapping	Rebuilds ml.item_modifiergroup_modifier_mapping (28 cols) — the ML-facing, denormalized modifier/menu-item mapping feature table, scoped by organization.
usp_refresh_menu_entities	Rebuilds ml.menu_entities (28 cols) — flattened menu-item + category features for the ML/DS team.
usp_refresh_modifier_impressions	Refreshes ml.modifier_impressions (33 cols) feature table; supports p_refresh_mode (full vs incremental).
usp_refresh_modifier_interactions	Refreshes ml.modifier_interactions (38 cols) feature table; supports p_refresh_mode (full vs incremental).

Procedure	Purpose
usp_refresh_transactions	Refreshes ml.transactions (37 cols) — order-grain feature table for downstream ML training/inference.
usp_refresh_upsell_analysis	Refreshes ml.upsell_analysis, aggregating upsell-recommendation acceptance signals.
usp_refresh_weather	Refreshes ml.weather (59 cols) — enriched weather-feature table joined to location/date for demand-modeling features.
usp_refresh_item_modifier_matching	Fuzzy-matches modifiers to menu items using dim.token_sort_ratio (ported from Python rapidfuzz) with a trigram prefilter (p_trgm_prefilter) and score threshold (p_tsr_threshold, default 70), writing results to ml.modifier_item_match, scoped per organization/location.

Seven of the eight follow a usp_refresh_<entity>(p_refresh_mode integer DEFAULT 1) signature — the refresh-mode parameter lets a caller choose full-rebuild vs. incremental refresh; check each procedure body for what mode 1 vs. other values actually does before assuming they mean the same thing across procedures — they were not necessarily written by the same convention at the same time as the dim.usp_refresh_* family.

5.3 Fuzzy Item–Modifier Matching Subsystem

ml.usp_refresh_item_modifier_matching(p_organizationid, p_tsr_threshold DEFAULT 70, p_trgm_prefilter DEFAULT 0.40) is the most algorithmically distinct procedure in the warehouse — everything else is SQL aggregation; this one implements fuzzy string matching directly in PL/pgSQL:

DELETE existing ml.modifier_item_match rows for the target org/location before rebuilding — the procedure is idempotent per scope, not append-only.

tmp_filtered_modifiers — collapses ml.item_modifiergroup_modifier_mapping to one row per modifierid, excluding modifier groups matched by name pattern ('%level%', '%change%', '%sauce choice%') and modifiers starting with 'no' — these are deliberately excluded because they are structural/negation modifiers, not candidates for item-name matching.

dim.token_sort_ratio(fm.modifiername, mi.menuitemname) — the core fuzzy-match function, ported from Python's rapidfuzz token_sort_ratio, computing a 0–100 similarity score between the two normalized name strings.

p_trgm_prefilter — a pg_trgm-based prefilter (similarity threshold, default 0.40) applied before the token_sort_ratio computation, to avoid running the expensive fuzzy comparison against every menu item for every modifier.

WHERE dim.token_sort_ratio(...) >= p_tsr_threshold — only matches scoring at or above the threshold (default 70) are written to ml.modifier_item_match, tagged with match_confidence_tier and match_direction.

FOLLOW-UP: dim.token_sort_ratio lives in the dim schema (it is a shared string-similarity utility, not ml-specific) but is only actually called from this one ml procedure today. If you need to review or modify the matching algorithm itself, look for CREATE FUNCTION dim.token_sort_ratio in the DDL export — it was not included in the nine files reviewed for this document and should be located and preserved separately if it is not already in dim_stored_procs.sql on the live server.

6. Schema: stg — Silver Staging Layer

45 tables — the landing zone between ADLS Bronze and the dim/fact/ml Gold schemas. Every stg table is populated by an ADF Copy Activity or Data Flow reading Bronze partition files, and every table in it is either fully truncated or window-deleted by `etl.truncate_silver_staging()` before the next run lands new data (Section 6.2) — nothing in stg is meant to accumulate indefinitely.

Dimension staging (loaded by ADF Copy/Data Flow from Cosmos DB, consumed by `dim.usp_refresh_*`)

Table	#Cols	Purpose
dim_catalog	13	
dim_category_hierarchy	28	
dim_cep_subscriptions	3	Raw CEP subscription payload staging (single JSON-ish column).
dim_frequentcustomer	12	
dim_itemcategory	15	
dim_kiosk	9	
dim_kiosk_appearance	7	Kiosk UI theme/appearance-config staging (receipt settings, fonts, style overrides).
dim_kiosk_config	30	Kiosk feature-flag/config staging (30 cols — scanners, ADA config, loyalty features, etc.).
dim_location_kiosks	8	
dim_loyalty_configuration	4	Per-location loyalty-provider configuration staging.
dim_menuitem	20	
dim_modifier	20	
dim_modifiergroup	20	
dim_modifiergroup_modifier_mapping	17	
dim_occasionsurvey	11	
dim_organization	33	
dim_organizationlocation	6	
dim_payment_provider	4	Per-location payment-provider staging.
dim_pos_provider	4	Per-location POS-provider staging.
kioskdetails	7	Legacy kiosk-details staging, superseded in Gold by <code>dim.kioskdetails</code> .

Device/infrastructure staging (full-truncate each run — see 6.2)

Table	#Cols	Purpose
fact_devicestate	14	
fact_devicetelemetry	8	
fact_itemssurvey	14	
fact_occasionsurveydetail	15	

Orders/transaction pipeline staging (10-minute sliding-window truncate — see 6.2)

Table	#Cols	Purpose
silver_all_gem_events	20	Raw GEM event stream staging before it is split into fact.deviceevent / fact.userbehaviour via usp_distribute_silver_gem_events.
silver_all_transaction_entities	52	Widest staging table (54 cols incl. Cosmos system fields _attachments/_etag/_rid/_self/_lsn/_ts) — raw camelCase mirror of the Cosmos order document (orderId, locationId, kioskSessionId, totals, items, combos, paymentDetails as nested JSON) plus a bronze_folderpath tracking column, before usp_distribute_silver_transaction_entities fans it out to the per-entity silver_transaction_* tables below.
silver_cep_incidents	22	CEP incident staging, feeds fact.usp_silver_cep_incidents_to_fact_cep_incidents.
silver_item_modifiers	52	Widest per-entity staging table (52 cols) — order-item-to-modifier staging.
silver_kiosk_events	22	Kiosk session/UI event staging — the events pipeline's primary landing table, joined by the transaction-header loader for journey timing (Section 4.3).
silver_modifier_impressions	31	Modifier-impression staging (31 cols).
silver_modifier_interactions	26	Modifier-interaction staging (26 cols).
silver_modifier_recommendations	20	Modifier-recommendation staging (20 cols).
silver_transaction_combo_items	54	Combo-item staging (widest transaction-entity staging table at 54 cols).
silver_transaction_header	78	Order-header staging (78 cols — the widest table in the entire warehouse), source for fact.usp_silver_transaction_header_to_fact (Section 4.3).
silver_transaction_item	40	Order line-item staging (40 cols).
silver_transaction_payment	37	Payment staging (37 cols).
silver_transaction_refunds	15	Refund staging.
silver_upsell_recommendations	23	Upsell-recommendation staging (23 cols).
gem_failed_order_job_notifications	15	Failed-job notification staging for operational monitoring.

Legacy / working / lookup tables

Table	#Cols	Purpose
lookup_silver_transaction_header	32	Lookup/reference copy of transaction-header staging shape (32 cols) — likely used for existence-check joins rather than direct loading.

Table	#Cols	Purpose
temp_silver_transaction_header	31	Working/scratch copy of the transaction-header staging shape (31 cols).
transactionheader	44	Older/alternate transaction-header staging shape (44 cols, snake_case, different column set than silver_transaction_header) — likely predates the current silver_* naming convention; confirm whether it is still written to before relying on it.
modifier_recommendation_sessions	13	Session-grain modifier-recommendation staging, feeds fact.modifier_recommendations.
recommendations	8	Item-recommendation staging.
sent_surveys	12	Survey-send event staging.

6.1 The camelCase Convention

stg.silver_all_transaction_entities is the one staging table that mirrors the raw Cosmos DB JSON document verbatim — its business columns are camelCase and double-quoted ("orderId", "locationId", "kioskSessionId", "businessDate", etc.) because they are direct 1:1 mirrors of the source JSON field names. This is the table the established convention refers to: ADF Copy Activity sink column names for this table (and the ARM template dataset stgSilverCaseSensitiveOrderEntitiesAnalyticsDb that maps to it) must match the PostgreSQL DDL column names exactly, character-for-character — never lowercased or snake_cased — or the sink mapping silently breaks.

Everything downstream of it (silver_transaction_header, silver_transaction_item, silver_transaction_payment, etc.) is flattened, parsed, and renamed to lowercase/snake_case by usp_distribute_silver_transaction_entities before landing — so the camelCase rule applies specifically to the raw landing table, not to the whole stg schema.

DISCREPANCY TO VERIFY: The ARM template still references a dataset/table named stg.silver_order_entities (dataset stgSilverCaseSensitiveOrderEntitiesAnalyticsDb, and a DELETE FROM stg.silver_order_entities WHERE "bronze_folderpath" = '@{item().partition_path}' cleanup statement inside BronzeOrdersToSilverAndGAS). That exact table name does not appear in the May 30, 2026 DDL export — only stg.silver_all_transaction_entities does, with a matching bronze_folderpath column. Most likely silver_order_entities was renamed to silver_all_transaction_entities and the ARM template simply has not been fully re-synced to match. Confirm which name is authoritative in production before touching either the ADF dataset or the table — this is exactly the kind of drift that is invisible until a pipeline run fails on a missing table.

7. Schema: etl — Pipeline Orchestration Metadata

One table: etl.bronze_partition_registry (17 cols) — the watermark ledger that lets ADF know which hourly Bronze partitions have already been processed, so re-running a pipeline never reprocesses a completed partition and a late-arriving hour is never silently skipped.

7.1 etl.bronze_partition_registry

Grain: one row per (entity, dateid) where dateid is an integer in YYYYMMDDHH24 form and entity is 'orders' or 'events'. Rows are pre-generated in bulk from dim.datedim for a future date range (the working queries show INSERT ... WHERE dt.yearval = 2026 AND dt.monthval IN (7,8,9,10,11,12) — i.e. partitions are seeded months ahead of time with status = 'pending') rather than created reactively as files arrive.

Column	Purpose
dateid	YYYYMMDDHH24 partition key — the primary lookup value pipelines filter on.
layer, entity	Always 'bronze' / 'orders' or 'events' today, but the columns leave room for future layers/entities.
partition_path	Full ADLS path, e.g. orders/raw/2026/07/15/23 — built by concatenating the dateid's year/month/day/hour substrings.
partition_date, _year, _month, _day, _hour	Denormalized date parts for fast filtering without re-parsing dateid.
status	'pending' → 'completed' or 'not found' — 'not found' means the hour had no source file (not an error, just nothing to load).
file_count, started_at, completed_at, adf_pipeline_run_id, error_message	Populated by the pipeline as it processes the partition — error_message is the first place to check when a partition is stuck at 'pending' past its expected window.

The PendingPartitions lookup used by the ADF ForEach loops (Section 13.5) queries this table for rows where status is not yet completed/not-found and dateid falls in the processable window — this is the mechanism that drives the parallel hour-partition read pattern.

7.2 etl.truncate_silver_staging()

Called once per DimensionsFactsMaster run, immediately after the Dimensions step and before BronzeSilverGASMaster loads new data (Section 13). Two different strategies, by table:

Full TRUNCATE — silver_transaction_payment, silver_item_modifiers, silver_upsell_recommendations, silver_modifier_recommendations, silver_modifier_interactions, silver_modifier_impressions, fact_devicestate, fact_devicetelemetry, fact_occasionsurveydetail, fact_itemssurvey, silver_cep_incidents, gem_failed_order_job_notifications — safe to clear completely because nothing downstream needs their prior-run contents.

10-minute sliding-window DELETE — silver_kiosk_events, silver_transaction_header, silver_transaction_item, silver_transaction_combo_items: DELETE WHERE syscosmosts < (watermark.ts - 600). The comment in the procedure explains why: up to 8 downstream stored procedures join this table by ordersessionid/token, and a kiosk session can start up to 10 minutes before its order completes (a kiosk auto-cancels a session after 10 minutes of inactivity, which is the effective upper bound). Also strips is_test_order = TRUE rows from the transaction staging tables.

DO NOT REVERT: silver_transaction_header, silver_transaction_item, and silver_transaction_combo_items are TRUNCATED inside this same procedure body via commented-out lines (--TRUNCATE TABLE stg.silver_transaction_header; etc.) directly above the DELETE statements that replaced them — a clear signal that the window-delete approach was a deliberate fix for a bug where a full truncate was destroying sessions that a same-run join still needed. Do not "clean up" those comments or revert to TRUNCATE for these three tables without understanding why the switch was made.

8. Sequences — Manual Surrogate Key Standard

gas_db does not use IDENTITY columns for surrogate keys. Every surrogate-keyed table uses a manually-created, manually-owned sequence, set up with the same five-step pattern (see `gas_db_sequence_synchronization.sql`):

```
CREATE SEQUENCE IF NOT EXISTS <schema>.<table>_<col>_seq — explicit START/INCREMENT/CACHE, no MIN/MAXVALUE.
```

```
ALTER SEQUENCE ... OWNER TO citus
```

```
ALTER SEQUENCE ... OWNED BY <schema>.<table>.<col> — ties the sequence's lifecycle to the column (dropped automatically if the column is ever dropped).
```

```
SELECT setval('<seq>', (SELECT COALESCE(MAX(<col>), 0) FROM <table>)) — synchronizes the sequence to the table's current max, safe on an empty table.
```

```
ALTER TABLE ... ALTER COLUMN <col> SET DEFAULT nextval('<seq>') — attaches it as the column default.
```

Two tables (`dim.view`, `dim.itemcategory`) explicitly run `ALTER TABLE ... ALTER COLUMN ... DROP IDENTITY IF EXISTS` first — meaning they were originally created with IDENTITY and were deliberately converted to this manual-sequence pattern. If you ever need to add a new surrogate-keyed table, follow this five-step pattern rather than using `GENERATED ... AS IDENTITY`, to stay consistent with the rest of the warehouse.

Sequence	Owning column
<code>dim.element_elementid_seq</code>	<code>dim.element_elementid_seq</code>
<code>dim.view_id_seq</code>	<code>dim.view_id_seq</code>
<code>dim.frequentcustomer_customerkey_seq</code>	<code>dim.frequentcustomer_customerkey_seq</code>
<code>dim.itemcategory_id_seq</code>	<code>dim.itemcategory_id_seq</code>
<code>dim.kiosk_id_seq</code>	<code>dim.kiosk_id_seq</code>
<code>dim.menuitem_id_seq</code>	<code>dim.menuitem_id_seq</code>
<code>dim.occasionsurvey_surveykey_seq</code>	<code>dim.occasionsurvey_surveykey_seq</code>
<code>dim.ordertype_id_seq</code>	<code>dim.ordertype_id_seq</code>
<code>fact.devicestate_id_seq</code>	<code>fact.devicestate_id_seq</code>
<code>fact.ordertiming_id_seq</code>	<code>fact.ordertiming_id_seq</code>
<code>fact.transactionheader_id_seq</code>	<code>fact.transactionheader_id_seq</code>
<code>fact.userbehaviour_id_seq</code>	<code>fact.userbehaviour_id_seq</code>

12 sequences total: 8 in dim (element, view, frequentcustomer, itemcategory, kiosk, menuitem, occasionsurvey, ordertype) and 4 in fact (devicestate, ordertiming, transactionheader, userbehaviour). Not every dim/fact table has a surrogate key sequence — several use a natural key (e.g. `modifierid`, `catalogid`) as their primary identifier instead; check the table's DDL before assuming a sequence exists.

9. Indexes

gas_indexes.sql defines 84 active index statements plus 18 that are present but commented out (either deliberately retired for zero usage, or staged for future creation and not yet applied). Understanding which is which matters before assuming index coverage.

Type	Active count	Notes
btree	53	Standard lookup/join indexes — the large majority.
BRIN	7	pages_per_range='128', all on syscosmosts (or createddate) — appropriate for append-mostly, time-correlated columns on very large fact tables.
GIN	0 (2 commented out)	Both commented-out GIN indexes target dim.menuentities JSON/array columns (mealavailability, tags) — not currently active.
UNIQUE	2	dim.itemcategory (locationid, categoryid) and dim.ordertype (locationid, kioskid, ordertypeid) — enforced via unique index rather than a table constraint.

9.1 BRIN Coverage — Known Gap

Active BRIN indexes exist on: fact.userbehaviour (createddate and syscosmosts — two separate BRIN indexes), fact.transactionheader (syscosmosts), fact.recommendations (syscosmosts), fact.itemmodifier (syscosmosts), fact.modifier_recommendations (syscosmosts), and stg.silver_kiosk_events (syscosmosts).

RECOMMENDED FOLLOW-UP: ix_deviceevent_syscosmosts_brin on fact.deviceevent — the largest fact table (262M+ rows) — is present in the file but commented out, meaning it has not been created. Given syscosmosts is exactly the kind of append-correlated column BRIN is built for, and every other comparably-sized fact table already has this index, this is very likely a leftover TODO rather than an intentional omission. Confirm with the team whether it was created directly in production after this file was last updated; if not, it is a strong first fix for the incoming owner.

9.2 Partial Indexes

ix_deviceevent_journey_lookup on fact.deviceevent is a partial btree index — (locationid, dateid, datacategory, eventtoken) WHERE datacategory = 'insight' AND actiontype IN (CategorySelected, SubCategorySelected, RegularItemSelected, ItemRemoved, ModifierGroupSelected, ModifierSelected, ModifierUnselected). This is the index the customer-journey/funnel queries against fact.deviceevent rely on — keep the WHERE predicate in sync if the set of "insight" actiontypes used for journey analysis ever changes, or journey queries will silently stop using the index.

10. Cross-Cutting Stored-Procedure Conventions

These patterns recur across all 64 procedures/functions regardless of schema. Recognizing them is the fastest way to read any procedure in the codebase without re-deriving it from scratch.

10.1 Definition & ownership boilerplate

```
CREATE OR REPLACE PROCEDURE <schema>.<name>(...)
  LANGUAGE plpgsql -- or 'sql' for pure set-based procedures
  AS $BODY$
  ...
```

```
$BODY$;
```

```
ALTER PROCEDURE <schema>.<name>(...) OWNER TO citus;
```

10.2 Watermark pattern

SELECT COALESCE(ts, <seed>) - <buffer> INTO v_max_syscosmosts FROM fact.watermarktable WHERE watermarktablename = '<table>' AND source = '<source>' — captured once at the top of the procedure, before any CTE runs, so the whole procedure execution uses one consistent cutoff. The buffer (600 seconds / 10 minutes is the most common value) exists to tolerate late-arriving records near an hour boundary. Note Section 4.3's finding: the watermark is not always used as a hard WHERE filter on the delta CTE anymore — after the cross-hour syscosmosts reliability bug, some loaders capture the watermark but no longer gate on it directly.

10.3 Dedup: DISTINCT ON over ROW_NUMBER()

SELECT DISTINCT ON (<natural_key_cols>) ... ORDER BY <natural_key_cols>, <recency_col> DESC is used everywhere in preference to ROW_NUMBER() OVER (...) WHERE rn = 1 — fewer lines, one less CTE layer, and Postgres can satisfy it with a single index-friendly sort rather than a full window-function pass.

10.4 Temp table + ANALYZE before merge

CREATE TEMP TABLE tmp_<x> ON COMMIT DROP AS SELECT ... followed immediately by ANALYZE tmp_<x>; before the temp table is used to drive a join or upsert. Without the ANALYZE, the planner has no statistics on the brand-new temp table and will often pick a poor join plan (e.g. nested loop over what should be a hash join) — this is a deliberate, load-bearing line, not boilerplate to trim.

10.5 Non-destructive upsert: ON CONFLICT DO UPDATE + COALESCE + guard

INSERT ... ON CONFLICT (<natural_key>) DO UPDATE SET col = COALESCE(target.col, EXCLUDED.col), ... WHERE (<at least one column would actually change>). The COALESCE ensures a later, partial run never overwrites a value a previous run already captured; the WHERE guard on the DO UPDATE avoids bumping updateddate / triggering downstream change-detection when nothing actually changed. See the full walkthrough in Section 4.3.

10.6 Manual sequence-based surrogate keys

nextval('<schema>.<table>_<col>_seq') is called explicitly in the INSERT column list — see Section 8. Never assume a table will auto-generate its id; if a sequence is out of sync (e.g. after a bulk restore), inserts will start colliding with existing ids rather than erroring cleanly, so check dim.<x>_id_seq vs. MAX(id) if you ever see unexpected ON CONFLICT behavior on a table that should have been a clean insert.

10.7 Session-parameter tuning for large-batch loops

Learned during the fact.deviceevent / fact.userbehaviour performance cycle (Section 4.4) and worth reusing on any future large-batch maintenance script:

- SET synchronous_commit = OFF; — biggest single win for loops doing many small commits; safe for ETL workloads where a replica-lag/crash-window data loss of the last few commits is acceptable (this is not appropriate for anything that must be durably committed before returning success to a caller).
- SET jit = OFF; — avoids JIT compilation overhead on many short, repeated statement executions inside a loop.

- SET work_mem = '1GB'; (or lower, e.g. 256MB, depending on workload) — set per session before a heavy dedup/rebuild run, not globally.

11. ADF Data Factory — Orchestration Layer Overview

The full ARM template export contains 67 pipelines, 80 mapping data flows, 198 datasets, 15 linked services, and 4 triggers. The table below groups all 67 pipelines by function; the full alphabetical list with resource type is in Appendix D.

Master orchestration (7)

DimensionsFactsMaster · BronzeSilverGASMaster · DimensionMaster · FactsMaster · GEM_GSH_GOU_Master · MasterStoredProcedures · ADfEventLoggingTemplate

Bronze ingestion (Cosmos DB → ADLS) (8)

IngestToBronze · CosmosDbChangeFeed · BronzeEventsToGAS · BronzeEventsToSilverAndGAS · BronzeOrdersToGAS · BronzeOrdersToSilverAndGAS · TestBronzeEventsToSilverAndGAS · TestBronzeOrdersToSilverAndGAS

Silver → Gold: orders & transactions (10)

SilverOrdersToGAS · SilverEventsToGAS · gasTransactionsToAnalyticsDb · gxsTransactionsToAnalyticsDb · TransactionPaymentRefunds · orderHeadersNoItemsToAnalyticsDb · OrderItemsNoHeadersStandAlone · AbortedItemsToPG · AbortedOrdersAndItemsToPG · DimensionUpdateOrdersToAnalyticsDb

Device & peripheral telemetry (3)

DevicesAndPeripherals · gshDeviceHealthToAnalyticsDbFactDeviceState · gshDeviceTelemetryToAnalyticsDb

GEM events & user behaviour (8)

GEMEventsToParquetsADLS · GEMUserBehaviourETL · GEMUserBehaviourEventsToPG · gemEventsToAnalyticsDb · gemUserBehaviourToAnalyticsDb · gemOrderTimingToAnalyticsDb · gemUserCheckedInToPG · gemCEPIncidents

Dimension refresh (Cosmos DB → dim/stg) (13)

DimensionABTests · DimensionFrequentCustomer · DimensionKiosks · DimensionMasterItemsAndModifiers · DimensionMenuItems · DimensionMenuModifiers · DimensionOrganizationLocationCatalog · DimensionWeatherInfo · ItemCategoriesCosmosToPG · KiosksCosmosToPG · MenusCosmosToPG · OrderTypesCosmosToPG · CopyGOUViewsToGAS

Recommendations, upsell & survey (3)

ModifierUpsellRecommendationsToGAS · UpsellRecommendationsToGAS · OccasionSurveyOrdersFeedback

ML training-data extraction (10)

ChildPipCompressedTrainingDataML · ChildPipTrainingDataML · CompressedTrainingDataForML · FastPartitionTrainingDataForML · SampleDataForMLTraining · SeparateDatasetsForMLTraining · TrainingDataForML · V2ChildPipTrainingDataML · V2TrainingDataForML · RefreshMLTables

Install-base dashboard (2)

GrubrrrInstallBaseDashboard · RefreshInstallBaseDataset

Maintenance / utility / test (3)

UpdateDateTimeFields · UpdateFieldsInTrxnFactTables · TestPipelines

11.1 Linked Services (15)

Name	Type
ADLSgas	AzureBlobFS
ADLSsus	AzureBlobFS
CosmosGXS	CosmosDb
CosmosNGE	CosmosDb
CosmosNGEGrubrrrLoyalty	CosmosDb

Name	Type
CosmosPOS	CosmosDb
EventsCosmosNew	CosmosDb
LoadKiosksFromCosmosDbToAnalyticsDb	AzureFunction
OrdersCosmosNew	CosmosDb
OrdersPGNew	AzurePostgreSql
PostgresGMS	AzurePostgreSql
PostgresGOU	AzurePostgreSql
ServicesPGNew	AzurePostgreSql
StgADLSgas	AzureBlobFS
kyv_grubrrr_dev	AzureKeyVault

Four Cosmos DB sources feed Bronze (CosmosGXS, CosmosNGE, CosmosNGEGrubrrrLoyalty, CosmosPOS, plus EventsCosmosNew/OrdersCosmosNew as newer variants); ADLSgas/ADLSsus/StgADLSgas are the three ADLS Gen2 (AzureBlobFS-typed) storage accounts; OrdersPGNew/PostgresGMS/PostgresGOU/ServicesPGNew are the PostgreSQL targets/sources; kyv_grubrrr_dev is the Key Vault backing linked-service credentials.

LINKED-SERVICE GOTCHA: All ADLS-linked services here are typed AzureBlobFS. This matters directly because of the stgsusprodeastus incident: a linked service pointed at a flat-namespace Blob Storage account but typed as ADLS Gen2 (dfs.core.windows.net) throws AdlsGen2OperationFailed / EndpointUnsupportedAccountFeatures. If a new storage account is ever added, confirm hierarchical namespace is actually enabled before typing its linked service as ADLS Gen2 / AzureBlobFS — otherwise use the plain Azure Blob Storage connector instead.

11.2 Triggers (4)

Trigger	Schedule	Fires pipeline
GASOrders	Daily @ 5:30 Eastern Standard Time	DimensionsFactsMaster
GEMEvents	Daily @ 5:00 Eastern Standard Time	GEM_GSH_GOU_Master
RefreshMLTables	Daily @ 11:00 Eastern Standard Time	RefreshMLTables
TwiceDaily	Daily @ 2,14:30 UTC	GrubrrrInstallBaseDashboard

POSSIBLE REDUNDANCY TO VERIFY: GASOrders triggers DimensionsFactsMaster once per day (5:30am EST) and GEMEvents triggers GEM_GSH_GOU_Master separately once per day (5am EST) — but GEM_GSH_GOU_Master is also called as a child step inside DimensionsFactsMaster itself (Section 12). That means GEM_GSH_GOU_Master can run twice in the same day through two independent triggers. This may be intentional (e.g. GEMEvents runs earlier to unblock something else before the main master pipeline kicks off), or it may be a legacy trigger left over from before GEM_GSH_GOU_Master was folded into the master pipeline. Confirm current intent with the team before disabling either trigger.

Because triggers fire once daily rather than hourly, "real time" in this warehouse means same-day batch, not streaming — the parallel hour-partition catch-up mechanism (Section 13) is what allows one daily run to process up to a full day's worth of pending hourly partitions in a single pipeline execution.

12. Master Orchestration: DimensionsFactsMaster

DimensionsFactsMaster is the single pipeline the GASOrders trigger fires once a day. Every other master/sub-pipeline in the warehouse is reached by following the ExecutePipeline chain out from here. Its 14 activities implement one linear success path plus two independent failure branches, both logging to fact.pipelinerunstatus.

12.1 Success Path

v_starttime — SetVariable, @utcNow()

Dimensions — ExecutePipeline → DimensionUpdateOrdersToAnalyticsDb (waitOnCompletion)

TruncateStagingTables — Lookup activity running CALL etl.truncate_silver_staging(); (Section 7.2)

BronzeSilverGASMaster — ExecutePipeline → BronzeSilverGASMaster, which itself runs BronzeEventsToSilverAndGAS and BronzeOrdersToSilverAndGAS

GEM_GSH_GOU_Master and Facts (→ FactsMaster) — both ExecutePipeline activities depend only on BronzeSilverGASMaster succeeding, so they run in parallel with each other *[parallel branch]*

v_finishtime — SetVariable, @utcNow(), depends on Facts succeeding

SuccessStatus — Lookup activity: UPDATE fact.pipelinerunstatus SET pipelinerunid=..., pipelinetriggertime=..., pipelinecompletedtime=..., issuccess mark

12.2 Failure Branches

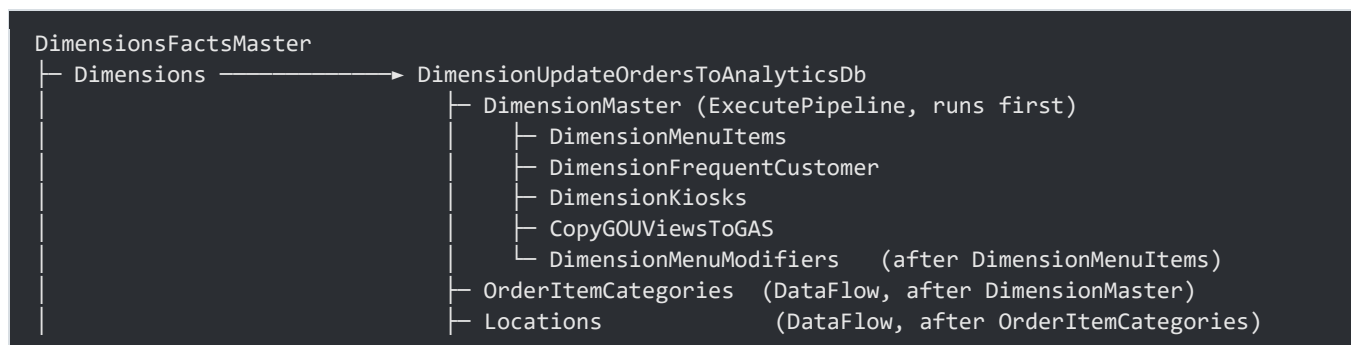
Two independent failure branches exist, each triggered by dependencyConditions: ['Failed'] on a specific step rather than by a try/catch construct (ADF pipelines don't have one) — this is the standard ADF pattern for error handling:

If Dimensions fails: v_errormsg captures @replace(activity('Dimensions').error.Message, "", "") → v_finishtime_copy1 → FailStatus (same UPDATE fact.pipelinerunstatus pattern, with the error message and a failure flag).

If Facts fails: the same three-step chain runs against activity('Facts').error.Message → v_errormsg2 → v_finishtime2 → FailStatus2.

MONITORING GAP: There is no failure branch keyed off BronzeSilverGASMaster or GEM_GSH_GOU_Master failing directly — only Dimensions and Facts have explicit error capture. If BronzeSilverGASMaster fails, the pipeline simply stops (Facts and GEM_GSH_GOU_Master never run, because their dependency was never satisfied) without ever writing a failure row to fact.pipelinerunstatus. Anyone monitoring pipeline health off fact.pipelinerunstatus should be aware a stuck/failed BronzeSilverGASMaster step will look like "no completion logged" rather than an explicit failure row — worth adding the missing failure branch if you find this pattern causing missed alerts.

12.3 Sub-Pipeline Call Tree



```

|
| NewCosmosOrdersMenus (DataFlow, after Locations)
| NgeCategoryHierarchy (DataFlow, after NewCosmosOrdersMenus)
| gshDeviceToDimKiosk (DataFlow, after NgeCategoryHierarchy)
| LoadOrderTypes (DataFlow, after gshDeviceToDimKiosk)
| 5x error-branch triples (v_errormsg/v_finishtime/FailStatus,
one per step group)
|
| TruncateStagingTables → CALL etl.truncate_silver_staging();
|
| BronzeSilverGASMaster → BronzeSilverGASMaster (pipeline)
|                         | BronzeEventsToSilverAndGAS
|                         | BronzeOrdersToSilverAndGAS
|                         | (both use the parallel hour-partition pattern – Section 13)
|
| GEM_GSH_GOU_Master → GEM_GSH_GOU_Master (pipeline)
|   dimView → dimElement → CopyGOUViewsToGAS →
|   gemEventsToAnalyticsDb → GEMUserBehaviourEventsToPG →
|   gshDeviceHealthToAnalyticsDbFactDeviceState →
|   gshDeviceTelemetryToAnalyticsDb → gemOrderTiming →
|   gemUserCheckedInToPG → DimensionABTests
|
| Facts → FactsMaster (pipeline)
|   dimOrderType (Lookup) → gasTransactionsToAnalyticsDb →
|   UpsellRecommendationsToGAS → ModifierUpsellRecommendationsToGAS →
|   AbortedOrdersAndItemsToPG → OccasionSurveyOrdersFeedback
|
| runs in
| parallel
| with
| Facts

```

Note the load order inside DimensionUpdateOrdersToAnalyticsDb: DimensionMaster (menu items, frequent customers, kiosks, modifiers, GOU views) runs to completion before OrderItemCategories/Locations/NewCosmosOrdersMenus/NgeCategoryHierarchy/gshDeviceToDimKiosk/LoadOrderTypes — the order-related dimension data flows all depend on the core dimensions being current first. Do not reorder these to "parallelize for speed" without checking for implicit read dependencies on freshly-refreshed dim tables.

13. Parallel Hour-Partition Read Pattern

BronzeOrdersToSilverAndGAS and BronzeEventsToSilverAndGAS (the two pipelines BronzeSilverGASMaster calls) both implement the same two-phase pattern for catching up on however many hourly Bronze partitions have accumulated since the last successful run.

13.1 Phase 1 — Parallel Existence Check

vTodayUTC — SetVariable, establishes the processing window's upper bound.

PendingPartitions — Lookup against etl.bronze_partition_registry: rows for this entity where status is not yet 'completed'/'not found' and dateid falls inside the eligible window (the working queries in etl_bronze_partition_registry.sql show the pattern: dateid > COALESCE(MAX(dateid) WHERE status IN ('completed','not found'), <seed>) AND dateid < NOW()-1hr, status = 'pending').

LoopThroughHourPartitions — ForEach over the Lookup output, isSequential: false, batchCount: 50. Up to 50 partitions are checked concurrently. Each iteration runs GetHourPartitions (GetMetadata — does this partition's ADLS folder actually exist?) then IfHourPartitionExists (IfCondition) to route existing vs. missing partitions.

13.2 Phase 2 — Sequential Load

LoopThroughTrxnHourPartitions / LoopThroughEventsHourPartitions — ForEach over the confirmed-available partitions, isSequential: true (no batchCount — one partition at a time). Each iteration: an ExecutePipeline call to SilverOrdersToGAS / SilverEventsToGAS, a GetMetadata + IfCondition existence check per source file type (header/item/payment/combo/modifiers/interactions/impressions/recommendations/refunds), then a Copy activity into the matching stg.silver_* table for every file type present.

DO NOT PARALLELIZE PHASE 2: Phase 1 (existence-checking) is parallel because it is cheap, read-only metadata calls. Phase 2 (actually copying data and executing the per-partition child pipeline) is deliberately sequential — this is the direct mitigation for the dimCategoryHierarchy SSL/connection-exhaustion issue (Npgsql "E" response to SSLRequest): running many Copy Activities against the same PostgreSQL source (GMS) in parallel exhausted available connections. If you are ever tempted to flip LoopThroughTrxnHourPartitions/LoopThroughEventsHourPartitions to isSequential: false for throughput, revisit that incident first — the other mitigations on record are replica reads, reduced parallelism, and watermark-based incremental partitioning, not parallel Phase 2 execution.

IngestToBronze (the pipeline one layer upstream of these two, landing raw Cosmos DB data into ADLS Bronze in the first place) is a much simpler linear chain: ngeOrdersToBronzeCopy → {ngeOrdersToBronze, gemEventsToBronze, OriginalNgeOrdersToBronze} → ngeLocationsToBronze. It does not use the hour-partition ForEach pattern — it is a straight Cosmos DB change-feed style pull, not a partition catch-up.

14. Known Issues, Landmines & Recommended Follow-ups

Consolidated from this document's analysis and recent working history. Ranked roughly by blast radius — items that would silently corrupt data or break production first, verification/documentation gaps second.

14.1 High Blast Radius

- `etl.truncate_silver_staging()` window logic — the 10-minute sliding-window DELETE on `silver_kiosk_events` / `silver_transaction_header` / `silver_transaction_item` / `silver_transaction_combo_items` must not be reverted to a full TRUNCATE (Section 7.2). This was a deliberate bug fix; reverting it will cause the transaction-header loader's kiosk-event journey-timing join to silently lose data again.
- `fact.usp_silver_transaction_header_to_fact`'s ON CONFLICT ... DO UPDATE COALESCE guards (Section 4.3) must not be loosened to unconditional overwrites — doing so would let a late partial re-run erase correctly-captured timing data from an earlier run.
- The ADF parallel hour-partition Phase 2 (Section 13.2) must stay `isSequential: true`. This is the fix for the `dimCategoryHierarchy` SSL/connection-exhaustion incident (Npgsql "E" response to `SSLRequest`, diagnosed as GMS-source connection exhaustion from parallel Copy Activity reads). Flipping it back to parallel for throughput would very likely reintroduce that failure.
- ADLS linked-service typing — always confirm hierarchical namespace is enabled before typing a new storage account's linked service as ADLS Gen2/AzureBlobFS (Section 11.1). Getting this wrong throws `AdlsGen2OperationFailed` / `EndpointUnsupportedAccountFeatures`, as it did for `stgsusprodeastus`.

14.2 Verify Against Production (DDL Export May Be Stale)

- `fact.deviceevent` unique constraint — not visible in the May 30, 2026 DDL export; confirm whether the dedup project's unique constraint (and any supporting index) exists in production (Section 4.4).
- `ix_deviceevent_syscosmosts_brin` — present in `gas_indexes.sql` but commented out (not created). Every comparably-sized fact table has this BRIN index already; confirm whether it was created directly in prod, and create it if not (Section 9.1).
- `stg.silver_order_entities` vs. `stg.silver_all_transaction_entities` — the ARM template still references the former by name (dataset `stgSilverCaseSensitiveOrderEntitiesAnalyticsDb`, and a DELETE FROM `stg.silver_order_entities` cleanup statement); only the latter exists in the current DDL export, with a matching `bronze_folderpath` column. Confirm which name is authoritative before modifying either the ADF dataset or the table (Section 6.1).
- `stg.transactionheader` — an older/alternate transaction-header staging shape (44 cols, different column set than `stg.silver_transaction_header`) still present in the DDL export. Confirm whether any active pipeline still writes to it before assuming it is dead.
- `dim.token_sort_ratio` — called from `ml.usp_refresh_item_modifier_matching` but its own CREATE FUNCTION definition was not present in any of the nine files reviewed here. Locate and preserve it explicitly (Section 5.3).
- The per-location `ROW_NUMBER()` dedup loop script used for the `fact.deviceevent`/userbehaviour cleanup (Section 4.4) was run ad hoc and is not part of the committed stored-procedure files. If it still exists anywhere, hand it off explicitly.

14.3 Known Data-Quality / Design Gaps

- Historically, zero BRIN coverage existed across roughly 35 tables on syscosmosts before the current 7-index BRIN rollout — worth re-auditing periodically as new large tables are added, since the pattern (append-heavy, time-correlated) recurs.
- Roughly 10 fact tables have no primary key at all (confirmed structurally by the very small number of PRIMARY KEY / UNIQUE constraints — 4 PKs and 2 unique indexes total — relative to 33 fact tables). Natural-key ON CONFLICT upserts still work without a formal PK/unique constraint as long as a matching unique index exists, but any fact table relying on ON CONFLICT without a backing unique index/constraint will fail outright the moment it is used — check before assuming a given fact table's loader can be safely modified to use ON CONFLICT.
- No foreign-key constraints exist anywhere in the DDL export (0 REFERENCES). This is a deliberate Citus/distributed-table tradeoff, not an oversight, but it does mean referential integrity between dim and fact tables is enforced only by ETL discipline (the LEFT JOINS in the loader procedures), not by the database itself.

14.4 Monitoring Gap

- DimensionsFactsMaster has explicit failure branches only for the Dimensions and Facts steps (Section 12.2). A failure in BronzeSilverGASMaster or GEM_GSH_GOU_Master stops the pipeline without writing a failure row to fact.pipelinerunstatus — anyone alerting off that table should know a missing row does not necessarily mean success.

15. Recommended KT Session Plan

Structured around how the incoming engineer will actually operate the system day to day, not around the order this document happens to present material in. Each session should leave behind a written artifact (this document plus any session-specific notes), since verbal KT does not survive contact with week three.

Session	Document sections	What to actually do
1. Architecture & mental model	Section 2 (medallion flow), Section 11–13 (ADF orchestration)	Walk the Cosmos → Bronze → Silver → Gold flow end to end, then trace one DimensionsFactsMaster run live in the ADF monitoring UI while narrating each branch of the call tree in Section 12.3.
2. The landmines session (highest value)	Section 14 in full	Go through every item in 14.1 and 14.2 explicitly. These are the things that take weeks to rediscover if undocumented — prioritize this over anything else if time is short.
3. Runbooks for recurring issues	Sections 9.1, 11.1, 13.2	SSL/connection-exhaustion diagnosis and mitigation playbook; ADLS linked-service typing checklist; how to read etl.bronze_partition_registry.status to find a stuck partition.
4. Live walkthrough: one full pipeline	Section 4.3 + Section 13	Trace one order from a raw Cosmos JSON document through stg.silver_all_transaction_entities → stg.silver_transaction_header → fact.transactionheader, showing every transformation and why the ON CONFLICT guard exists.
5. Shadow / Q&A	Whole document	Incoming engineer drives (makes a small, low-risk change or runs a diagnostic query) while narrating their own understanding back — this surfaces gaps in comprehension faster than more explanation would.

16. Appendices

Appendix A — Full ADF Pipeline Inventory (67, alphabetical)

Pipeline	Pipeline
ADFEventLoggingTemplate	AbortedItemsToPG
AbortedOrdersAndItemsToPG	BronzeEventsToGAS
BronzeEventsToSilverAndGAS	BronzeOrdersToGAS
BronzeOrdersToSilverAndGAS	BronzeSilverGASMaster
ChildPipCompressedTrainingDataML	ChildPipTrainingDataML
CompressedTrainingDataForML	CopyGOUViewsToGAS
CosmosDbChangeFeed	DevicesAndPeripherals
DimensionABTests	DimensionFrequentCustomer
DimensionKiosks	DimensionMaster
DimensionMasterItemsAndModifiers	DimensionMenuItems
DimensionMenuModifiers	DimensionOrganizationLocationCatalog
DimensionUpdateOrdersToAnalyticsDb	DimensionWeatherInfo
DimensionsFactsMaster	FactsMaster
FastPartitionTrainingDataForML	GEMEventsToParquetsADLS
GEMUserBehaviourETL	GEMUserBehaviourEventsToPG
GEM_GSH_GOU_Master	GrubrrrInstallBaseDashboard
IngestToBronze	ItemCategoriesCosmosToPG
KiosksCosmosToPG	MasterStoredProcedures
MenusCosmosToPG	ModifierUpsellRecommendationsToGAS
OccasionSurveyOrdersFeedback	OrderItemsNoHeadersStandAlone
OrderTypesCosmosToPG	RefreshInstallBaseDataset
RefreshMLTables	SampleDataForMLTraining
SeparateDatasetsForMLTraining	SilverEventsToGAS
SilverOrdersToGAS	TestBronzeEventsToSilverAndGAS
TestBronzeOrdersToSilverAndGAS	TestPipelines
TrainingDataForML	TransactionPaymentRefunds
UpdateDateTimeFields	UpdateFieldsInTrxnFactTables
UpsellRecommendationsToGAS	V2ChildPipTrainingDataML
V2TrainingDataForML	gasTransactionsToAnalyticsDb
gemCEPIncidents	gemEventsToAnalyticsDb

Pipeline	Pipeline
gemOrderTimingToAnalyticsDb	gemUserBehaviourToAnalyticsDb
gemUserCheckedInToPG	gshDeviceHealthToAnalyticsDbFactDeviceState
gshDeviceTelemetryToAnalyticsDb	gxsTransactionsToAnalyticsDb
orderHeadersNoItemsToAnalyticsDb	

Appendix B — Source File Cross-Reference

Which of the nine uploaded artifacts backs each part of this document, for whoever needs to go re-verify something against the raw source rather than this summary.

Source file	What it contains	Covered in
gas_db_ddls_merged_20260530.sql	Full DDL for all 128 tables across dim/fact/ml/stg/etl.	Sections 3–7
gas_indexes.sql	All 84 active + 18 commented-out index/constraint statements.	Section 9
gas_db_sequence_synchronization.sql	The 12 manually-managed surrogate-key sequences.	Section 8
dim_stored_procs.sql	20 dim schema stored procedures.	Section 3
fact_stored_procs.sql	34 fact schema procedures + 2 utility functions.	Section 4
ml_stored_procs.sql	8 ml schema stored procedures.	Section 5
etl_bronze_partition_registry.sql	etl.bronze_partition_registry DDL + working watermark queries.	Section 7.1
silver_usp_etl_truncate_silver_staging.sql	etl.truncate_silver_staging() procedure body.	Section 7.2
ARMTemplateForFactory.json	Full ADF ARM export: 67 pipelines, 80 data flows, 198 datasets, 15 linked services, 4 triggers.	Sections 11–13

Appendix C — Document Change Log

Date	Change
July 2026	Initial version, generated from the nine source files listed above ahead of end-of-July departure KT sessions.